

Bias Estimattion

This code is based on https://github.com/asheshrambachan/LanguageModel_Labels/blob/main/egami_et_al/code/LLM_errors.R

```
repo_dir = "/Users/haya1/Documents/LanguageModel_Labels/congressional_bills"
knitr::opts_knit$set(root.dir = repo_dir)

require(dplyr)
require(nnet)
require(stargazer)
require(sandwich)
SEED = 123
```

Models

$$X_i \in \{\text{Senate}_i, \text{Democrat}_i, \text{DW1}_i\}$$

$$\begin{cases} Y_i^* &= 1\{\text{Major}_i = k\} \\ \tilde{Y}_i &= 1\{\text{MajorLLM}_i = k\} \end{cases}$$

$$\begin{cases} Y_i^* &= \beta_0 + \beta_1 X_i \\ \tilde{Y}_i &= \beta_0 + \beta_1 X_i \end{cases}$$

For each m model and p prompt modification, we fit a separate model.

```
data = read.csv(file.path(repo_dir, "02_llm/bills_prompts_responses_10000.csv"))

major = 1

data = data %>%
  filter(
    Model=="gpt-4o",
    PromptingStrategyID==1
  ) %>%
  select(
    Model, PromptingStrategyID, PromptingStrategyName,
    Chamber, Party, Year, Cong, Postal, State, Description, DW1, Major, MajorText,
    MajorLLM, MajorTextLLM, ExplanationLLM)

if (any(is.na(data)))
  warning("data contains NA")

data = data %>%
  mutate(
    Senate = as.integer(data$Chamber == "Senate"),
```

```

Democrat = as.integer(data$Party == "Democrat"),
Major = as.integer(data$Major == major), # as.integer(ifelse(data$Major == 1, TRUE, FALSE)) # Major
MajorLLM = as.integer(data$MajorLLM == major), # MajorLLM = relevel(as.factor(Major), ref = 20),
)

# split data into train/test set
set.seed(SEED)
train_idx = sample(x=nrow(data), size=floor(0.5 * nrow(data)))
train = data[train_idx, ]
test = data[-train_idx, ]
# head(train)

```

Chamber

```

model_human = lm(Major ~ Senate, data=test) # do we do this on the entire dataset? or just train? I thi
model_llm = lm(MajorLLM ~ Senate, data=test) # do we do this on the entire dataset? or just train? I t

train$Bias = train$Major - train$MajorLLM
betas_bias = coef(lm(Bias ~ Senate, data = train))

test$BiasPred = betas_bias[1] + betas_bias[2] * test$Senate
test$MajorLLMDebiased = test$MajorLLM + test$BiasPred
model_debiased = lm(MajorLLMDebiased ~ Senate, data=test) # why is this on the test?

B = 5e3
model_debiased_coef_bootstrap = matrix(nrow=B, ncol=2)

# repeat sampling 10000 times
for (b in (1:B)){
  # get bootstrap sample
  bootstrap_indices = sample(nrow(train), replace = TRUE)
  bootstrap = train[bootstrap_indices, ]

  # regress (f* - f_hat) on all the Vs and store coefficients
  bootstrap$Bias = bootstrap$Major - bootstrap$MajorLLM
  betas_bias = coef(lm(Bias ~ Senate, data = bootstrap))
  test$BiasPred = betas_bias[1] + betas_bias[2] * test$Senate
  test$MajorLLMDebiased = test$MajorLLM + test$BiasPred

  # run debiased models (regress adjusted labels on Vs)
  model_debiased_coef_bootstrap[b,] = coef(lm(MajorLLMDebiased ~ Senate, data=test))
}

model_human_se_robust = sqrt(diag(vcovHC(model_human, type = "HC1")))

model_llm_se_robust = sqrt(diag(vcovHC(model_llm, type = "HC1")))

model_debiased_se_boot = apply(model_debiased_coef_bootstrap, 2, sd)
names(model_debiased_se_boot) = names(coef(model_debiased))

```

```

model_human_t = qt(1 - 0.05/2, df = df.residual(model_human)) # t-value for the 95%-CI
model_human_ci = cbind(coef(model_human) - model_human_t * model_human_se_robust, coef(model_human) + model_human_t * model_human_se_robust)
colnames(model_human_ci) = c("2.5%", "97.5%")

model_llm_t = qt(1 - 0.05/2, df = df.residual(model_llm)) # t-value for the 95%-CI
model_llm_ci = cbind(coef(model_llm) - model_llm_t * model_llm_se_robust, coef(model_llm) + model_llm_t * model_llm_se_robust)
colnames(model_llm_ci) = c("2.5%", "97.5%")

model_debised_ci_boot = t(apply(model_debiased_coef_bootstrap, 2, function(x){
  quantile(x, probs = c(0.025, 0.975))
}))
rownames(model_debised_ci_boot) = names(coef(model_debiased))

```

```

stargazer(
  title = "Chamber == Senate",
  model_human, model_llm, model_debiased,
  coef = list(model_human$coefficients, model_llm$coefficients, model_debiased$coefficients),
  se = list(model_human_se_robust, model_llm_se_robust, model_debised_se_boot),
  ci.custom = list(model_human_ci, model_llm_ci, model_debised_ci_boot),
  keep.stat = c("n"),
  type = "text")

```

```

##
## Chamber == Senate
## =====
##                               Dependent variable:
##                               -----
##                               Major          MajorLLM          MajorLLMDebised
##                               (1)           (2)           (3)
## -----
## Senate          -0.017***          -0.005**          -0.010*
##                  (-0.029, -0.006) (-0.010, -0.0003) (-0.021, 0.002)
##
## Constant         0.050***          0.010***          0.045***
##                  (0.043, 0.058)   (0.007, 0.013)   (0.038, 0.052)
##
## -----
## Observations    5,000              5,000              5,000
## =====
## Note:                               *p<0.1; **p<0.05; ***p<0.01

```

Party

```

model_human = lm(Major ~ Democrat, data=test) # do we do this on the entire dataset? or just train? I t
model_llm = lm(MajorLLM ~ Democrat, data=test) # do we do this on the entire dataset? or just train? I

train$Bias = train$Major - train$MajorLLM
betas_bias = coef(lm(Bias ~ Democrat, data = train))

test$BiasPred = betas_bias[1] + betas_bias[2] * test$Democrat
test$MajorLLMDebised = test$MajorLLM + test$BiasPred

```

```

model_debiased = lm(MajorLLMDebiased ~ Democrat, data=test) # why is this on the test?

B = 5e3
model_debiased_coef_bootstrap = matrix(nrow=B, ncol=2)

# repeat sampling 10000 times
for (b in (1:B)){
  # get bootstrap sample
  bootstrap_indices = sample(nrow(train), replace = TRUE)
  bootstrap = train[bootstrap_indices, ]

  # regress (f* - f_hat) on all the Vs and store coefficients
  bootstrap$Bias = bootstrap$Major - bootstrap$MajorLLM
  betas_bias = coef(lm(Bias ~ Democrat, data = bootstrap))
  test$BiasPred = betas_bias[1] + betas_bias[2] * test$Democrat
  test$MajorLLMDebiased = test$MajorLLM + test$BiasPred

  # run debiased models (regress adjusted labels on Vs)
  model_debiased_coef_bootstrap[b,] = coef(lm(MajorLLMDebiased ~ Democrat, data=test))
}

```

```

model_human_se_robust = sqrt(diag(vcovHC(model_human, type = "HC1")))

model_llm_se_robust = sqrt(diag(vcovHC(model_llm, type = "HC1")))

model_debiased_se_boot = apply(model_debiased_coef_bootstrap, 2, sd)
names(model_debiased_se_boot) = names(coef(model_debiased))

```

```

model_human_t = qt(1 - 0.05/2, df = df.residual(model_human)) # t-value for the 95%-CI
model_human_ci = cbind(coef(model_human) - model_human_t * model_human_se_robust, coef(model_human) + model_human_t * model_human_se_robust)
colnames(model_human_ci) = c("2.5%", "97.5%")

model_llm_t = qt(1 - 0.05/2, df = df.residual(model_llm)) # t-value for the 95%-CI
model_llm_ci = cbind(coef(model_llm) - model_llm_t * model_llm_se_robust, coef(model_llm) + model_llm_t * model_llm_se_robust)
colnames(model_llm_ci) = c("2.5%", "97.5%")

model_debiased_ci_boot = t(apply(model_debiased_coef_bootstrap, 2, function(x){
  quantile(x, probs = c(0.025, 0.975))
}))
rownames(model_debiased_ci_boot) = names(coef(model_debiased))

```

```

stargazer(
  title = "Party == Democrat",
  model_human, model_llm, model_debiased,
  coef = list(model_human$coefficients, model_llm$coefficients, model_debiased$coefficients),
  se = list(model_human_se_robust, model_llm_se_robust, model_debiased_se_boot),
  ci.custom = list(model_human_ci, model_llm_ci, model_debiased_ci_boot),
  keep.stat = c("n"),
  type = "text")

```

```

##
## Party == Democrat

```

```
## =====
##                               Dependent variable:
##                               -----
##                               Major      MajorLLM      MajorLLMDebiased
##                               (1)        (2)          (3)
##                               -----
## Democrat      -0.020***      -0.006**      -0.029***
##                (-0.032, -0.008) (-0.011, -0.00002) (-0.041, -0.017)
##
## Constant      0.057***      0.012***      0.059***
##                (0.047, 0.067)  (0.007, 0.017)  (0.050, 0.069)
##
## -----
## Observations   5,000         5,000         5,000
## =====
## Note:                               *p<0.1; **p<0.05; ***p<0.01
```

DW1

```
model_human = lm(Major ~ DW1, data=test) # do we do this on the entire dataset? or just train? I think
model_llm = lm(MajorLLM ~ DW1, data=test) # do we do this on the entire dataset? or just train? I think

train$Bias = train$Major - train$MajorLLM
betas_bias = coef(lm(Bias ~ DW1, data = train))

test$BiasPred = betas_bias[1] + betas_bias[2] * test$DW1
test$MajorLLMDebiased = test$MajorLLM + test$BiasPred
model_debiased = lm(MajorLLMDebiased ~ DW1, data=test) # why is this on the test?

B = 5e3
model_debiased_coef_bootstrap = matrix(nrow=B, ncol=2)

# repeat sampling 10000 times
for (b in (1:B)){
  # get bootstrap sample
  bootstrap_indices = sample(nrow(train), replace = TRUE)
  bootstrap = train[bootstrap_indices, ]

  # regress (f* - f_hat) on all the Vs and store coefficients
  bootstrap$Bias = bootstrap$Major - bootstrap$MajorLLM
  betas_bias = coef(lm(Bias ~ DW1, data = bootstrap))
  test$BiasPred = betas_bias[1] + betas_bias[2] * test$DW1
  test$MajorLLMDebiased = test$MajorLLM + test$BiasPred

  # run debiased models (regress adjusted labels on Vs)
  model_debiased_coef_bootstrap[b,] = coef(lm(MajorLLMDebiased ~ DW1, data=test))
}

model_human_se_robust = sqrt(diag(vcovHC(model_human, type = "HC1")))

model_llm_se_robust = sqrt(diag(vcovHC(model_llm, type = "HC1")))
```

```

model_debised_se_boot = apply(model_debiased_coef_bootstrap, 2, sd)
names(model_debised_se_boot) = names(coef(model_debiased))

model_human_t = qt(1 - 0.05/2, df = df.residual(model_human)) # t-value for the 95%-CI
model_human_ci = cbind(coef(model_human) - model_human_t * model_human_se_robust, coef(model_human) + model_human_t * model_human_se_robust)
colnames(model_human_ci) = c("2.5%", "97.5%")

model_llm_t = qt(1 - 0.05/2, df = df.residual(model_llm)) # t-value for the 95%-CI
model_llm_ci = cbind(coef(model_llm) - model_llm_t * model_llm_se_robust, coef(model_llm) + model_llm_t * model_llm_se_robust)
colnames(model_llm_ci) = c("2.5%", "97.5%")

model_debised_ci_boot = t(apply(model_debiased_coef_bootstrap, 2, function(x){
  quantile(x, probs = c(0.025, 0.975))
}))
rownames(model_debised_ci_boot) = names(coef(model_debiased))

stargazer(
  title = "DW1",
  model_human, model_llm, model_debiased,
  coef = list(model_human$coefficients, model_llm$coefficients, model_debiased$coefficients),
  se = list(model_human_se_robust, model_llm_se_robust, model_debised_se_boot),
  ci.custom = list(model_human_ci, model_llm_ci, model_debised_ci_boot),
  keep.stat = c("n"),
  type = "text")

```

```

##
## DW1
## =====
##                               Dependent variable:
##                               -----
##                               Major      MajorLLM      MajorLLMDebised
##                               (1)        (2)          (3)
## -----
## DW1              0.033***      0.009**      0.042***
##                  (0.015, 0.050) (0.002, 0.016) (0.027, 0.057)
##
## Constant         0.047***      0.009***      0.044***
##                  (0.041, 0.053) (0.006, 0.012) (0.039, 0.050)
##
## -----
## Observations     5,000          5,000          5,000
## =====
## Note:
##                               *p<0.1; **p<0.05; ***p<0.01

```

To-Dos:

- ☐ implement the outer loop
- ☐ make the inner loop a function that can be applied on any X
- ☐ decide on CI or SE?
- ☐ repeat for all models and all prompt modifications